CS 327: Compilers
Assignment A2: Lexical Analyzer (Language Specification)

Swayam Borate (23110066) Shardul Junagade (23110297) Tejas Lohia (23110335)

06 February 2026

1 Overview

This document specifies the source language for our group compiler project. For Assignment A2, the main deliverable is the lexical analyzer; this PDF is our language-specification write-up for the same. The intent is to keep the language close to a small, C-like core so that it remains implementable using `lex` and `yacc` in C.

As a reference starting point, we studied the provided *YAPL* specification (a C-like teaching language). Our language is *adapted from that reference* in the sense that we keep the same overall style (C-like syntax, expressions, and statements). On top of the baseline, we add a few small extensions that we proposed and confirmed.

Phase focus. Since A2 focuses on scanning, this specification emphasizes tokens, literals, operators, and comment rules. We include a small syntax sketch only where it helps remove ambiguity during tokenization. Full parsing details will be handled in later phases.

2 Design goals

Goals

- A C-like language with familiar declarations, expressions, and control flow.
- A clean token set suitable for a `lex` scanner and a future `yacc` parser.
- Predictable handling of comments, strings, and numeric literals.

3 Baseline (YAPL-like) feature set

Unless stated otherwise in later sections, the baseline language includes the following C-like constructs.

Declarations and definitions

- Global variables (including `extern` declarations).
- Function declarations (prototypes) and function definitions (with bodies), including recursion.
- Local declarations inside blocks; multiple declarators separated by commas.
- `struct` types and field access using `.` and `->`.

Statements

- Selection: `if` / `else` ladders; `switch` / `case` with optional `default` and fall-through.
- Iteration: `for`, `while`, and `do-while` loops.
- Jump: `break`, `continue`, `return`.
- Expression statements ending with a semicolon.

Literals

- Integer literals (decimal and hexadecimal of the form `0x...`).
- Floating-point literals.
- Character literals and string literals.
- Adjacent string literals are treated as concatenated (C-style), e.g., `"a" "b"`.

Operators (C-like precedence)

The baseline keeps the usual precedence/associativity of C for the supported operators, and adds a three-way comparison operator `<=>`.

4 Extensions proposed by our team

In addition to the baseline, we add the following features.

4.1 `elif` as a replacement for `else if`

We support `elif` as syntactic sugar inside an if-ladder. It behaves like `else if`.

Example:

```
if (x < 0) {
        print("neg");
} elif (x == 0) {
        print("zero");
} else {
        print("pos");
}
```

4.2 `pass` as an empty statement

We support `pass;` as an explicit no-op statement. This is equivalent to an empty statement `;`, but is easier to read in some contexts.

4.3 Built-ins `print(...)` and `range(...)`

We treat `print` and `range` as built-in functions. For the compiler front-end, this mainly means they are reserved words and have fixed call syntax.

- `print(expr_list?)`: prints its arguments; the exact runtime behavior is out of scope.
- `range(l, r)`: produces an integer sequence used by the restricted loop form described next.

4.4 Restricted Python-style loop: `for (i in range(l, r))`

We add a specific loop form:

```
for (i in range(l, r)) stmt
```

Constraints (kept intentionally narrow for clean parsing):

- The loop variable must be an identifier (e.g., `i`).
- The iterator expression must be exactly `range(l, r)` with two expressions.
- No step argument is supported in `range` for this loop form.

Intended meaning (informal): iterate with `i` taking values from `l` up to but not including `r` (half-open interval like Python). This meaning is informational; for A2/A3 we only require correct scanning and parsing.

4.5 `#` as a single-line comment

In addition to `//` and `/* ... */`, we support `#` as a single-line comment starter. Since preprocessors are not part of the language, `#` is never treated as a directive.

4.6 `try-except` exception handling

We add Python-like exception handling using C-style blocks. The minimal supported form is:

```
try { ... } except { ... }
```

Notes:

- Only a single `except` clause is required.
- We do not require exception types, raising, or matching in A2/A3.

4.7 Optional operator extensions (only if clean)

We are considering two additional operators as optional extensions. They are not required for the base submission if they complicate precedence/lexing.

- `**` (exponent): right-associative; higher precedence than `* / %`.
- `??` (null-coalescing): returns the left operand if it is non-null, otherwise the right operand. For parsing, it can be treated as right-associative with precedence between logical-or (`||`) and assignment (`=`).

5 Lexical structure

5.1 Whitespace

Spaces, tabs, and newlines separate tokens and are otherwise ignored, except that newlines terminate single-line comments.

5.2 Identifiers

Identifiers follow the usual C rule:

- start with a letter or underscore;
- continue with letters, digits, or underscores.

In regular-expression form:

```
[A-Za-z_][A-Za-z0-9_]*
```

5.3 Reserved keywords

The following words are reserved and cannot be used as identifiers.

Core keywords.

`int`	`long`	`short`	`char`
`float`	`double`	`void`	`struct`
`extern`	`return`	`break`	`continue`
`if`	`else`	`switch`	`case`
`default`	`for`	`while`	`do`

Extension keywords.

`elif`	`pass`	`try`	`except`
`print`	`range`	`in`	

Any standard C keyword *not* listed above is treated as a normal identifier in our language. This keeps the keyword list small and helps with scanning.

5.4 Numeric literals

Integer literals. We support:

- decimal: `0` or `[1-9][0-9]*`
- hexadecimal: prefix `0x` followed by one or more hex digits `[0-9a-fA-F]+`

Floating-point literals. We support typical decimal floats such as `3.14`, `0.5`, and `10.`. Scientific notation may be added later if needed.

5.5 Character and string literals

Character literals. Single-quoted characters like `'a'` are supported. Common escape sequences may be supported (e.g., `\n`, `\t`, `\
`, `\'`).

String literals. Double-quoted strings like `"hello"` are supported, with escape sequences similar to character literals. Adjacent strings are concatenated:

```
"hello" "world"   // treated as one string
```

5.6 Comments

- **Single-line (C++ style):** `//` continues up to the end of the line.
- **Single-line (extension):** `#` continues up to the end of the line.
- **Multi-line:** `/*` begins a comment that ends at the next `*/`.

If the input ends while inside a multi-line comment, the lexer must treat it as an error (ill-formed/unclosed comment).

6 Operators and punctuators

Table 1 lists the main operators and separators. Where we say "C precedence", it refers to the standard precedence and associativity used in C for the listed operators.
Not supported (by design): preprocessors/directives, ternary `?:`, compound assignments (`+=`, `-=`, ...), shift operators (`«`, `»`), and varargs/ellipsis (`...`).

Category	Tokens
Arithmetic	`+ - * / %`
Relational	`< > <= >= == != <=>`
Logical	`&& || !`
Bitwise (subset)	`& | ^ ~`
Unary / special	`sizeof`
Assignment	`=`
Inc/Dec	`++ -`
Pointer / address	`* &`
Field access	`. ->`
Calls / indexing	`( ) [ ]`
Separators	`, ; :`
Grouping	`{ }`

Table 1: Operators and punctuators

7 Syntax sketch (informal)

This section is not a full grammar, but it captures the main statement forms.

- **Compound statement:**

  `{ decl* stmt* }`
- **Expression statement:**

  `expr ;`
- **Empty statement:**

  `;` or `pass ;`
- **If ladder:**

  `if (expr) stmt (elif (expr) stmt)* (else stmt)?`
- **Loops:**

  `while (expr) stmt`

  `do stmt while (expr) ;`

  `for (init? ; cond? ; step?) stmt`

  `for (id in range(expr, expr)) stmt`
- **Switch:**

  `switch (expr) { (case const : stmt*)* (default : stmt*)? }`
- **Try/except:**

  `try { stmt* } except { stmt* }`
- **Jump:**

  `break ;`

  `continue ;`

  `return expr? ;`